Exhaustiveness Checking

Concepts of Programming Languages

CAS CS 320

Recap

Pattern Matching (High Level)

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

Values come in different "shapes"

We want to do things based on the different shapes

Matching allows us to condition on the shape of the value

Patterns (High Level)

```
match x with
| 1 -> e1
| 2 -> e2
| _ -> e3
```

A pattern describes the "shape" of a **value**

But we don't *just* care about the "shape" we also want to do stuff with the pieces of the data

<u>Patterns</u> are syntactic representations of values with names

Shallow vs. Deep Pattern Matching

```
let third l =
  match l with
  | _ :: _ :: x :: _ -> Some x
  | _ -> None
```

We're interested implementing *deep* pattern matching, which allows for *nested* patterns

We can also implement *shallow* pattern matching ADTs (which are sometimes called *eliminators*)

Match Syntax

```
<expr> ::= match <expr> with <arms>
<arms> ::= <arm> <arms> | <arm>
 <arm> ::= | <pat> -> <expr>
 <pat> ::= () | true | false
         | [] | <pat> :: <pat>
         | <pat> , <pats>
         | <int> | <var> | ( <pat> )
<pats> ::= <pat> , <pats> | <pat>
```

Pattern Typing Judgments

$$p : \tau \Rightarrow \Gamma$$

"The pattern p is of type τ and generates the variable declarations Γ"

Our derivations now have multiple kinds of judgments

Match Typing

$$\frac{}{\texttt{()} : \texttt{unit} \Rightarrow \varnothing}\ \textsc{UnitP} \qquad \frac{}{\texttt{true} : \texttt{bool} \Rightarrow \varnothing}\ \textsc{TrueP} \qquad \frac{}{\texttt{false} : \texttt{bool} \Rightarrow \varnothing}\ \textsc{FalseP}$$

$$\frac{\text{n is an integer literal pattern}}{\mathsf{n} : \texttt{int} \Rightarrow \varnothing}\ \textsc{IntLitP} \qquad \frac{x \text{ is a variable pattern}}{x : \tau \Rightarrow \{x : \tau\}}\ \textsc{VarP} \qquad \frac{}{\texttt{[]} : \texttt{int list} \Rightarrow \varnothing}\ \textsc{NilP}$$

$$\frac{p_1 : \texttt{int} \Rightarrow \Gamma_1 \qquad p_2 : \texttt{int list} \Rightarrow \Gamma_2 \qquad \Gamma_1 \text{ and } \Gamma_2 \text{ are disjoint}}{p_1 \texttt{ :: } p_2 : \texttt{int list} \Rightarrow \Gamma_1, \Gamma_2}\ \textsc{ConsP}$$

$$\frac{p_1 : \tau_1 \Rightarrow \Gamma_1 \qquad \ldots \qquad p_k : \tau_k \Rightarrow \Gamma_k \qquad \Gamma_1, \ldots, \Gamma_k \text{ are disjoint}}{p_1 \texttt{ , } \ldots \texttt{ , } p_k : \tau_1 \texttt{ * } \ldots \texttt{ * } \tau_k \Rightarrow \Gamma_1, \ldots, \Gamma_k}\ \textsc{TupleP}$$

$$\frac{\Gamma \vdash e_0 : \tau_0 \qquad p_1 : \tau_0 \Rightarrow \Gamma_1 \qquad \Gamma, \Gamma_1 \vdash e_1 : \tau \qquad \ldots \qquad p_k : \tau_0 \Rightarrow \Gamma_k \qquad \Gamma, \Gamma_k \vdash e_k : \tau}{\Gamma \vdash \texttt{match } e_0 \texttt{ with } \texttt{|}\ p_1 \texttt{ -> } e_1 \ldots \texttt{ |}\ p_k \texttt{ -> } e_k : \tau}\ \textsc{Match}$$

Pattern matching judgment

$$v \text{ matches } p \Rightarrow \mathscr{E}$$

"The value v matches the pattern p and generates the bindings $\mathscr{E}$"

Again, our derivations now have multiple kinds of judgments

Match Semantics

$$\frac{}{\texttt{()} \text{ matches } \texttt{()} \Rightarrow \varnothing}\ \textsc{unitM} \qquad \frac{}{\top \text{ matches } \texttt{true} \Rightarrow \varnothing}\ \textsc{trueM} \qquad \frac{}{\bot \text{ matches } \texttt{false} \Rightarrow \varnothing}\ \textsc{falseM}$$

$$\frac{\text{n is an integer pattern for } n}{n \text{ matches n} \Rightarrow \varnothing}\ \textsc{intM} \qquad \frac{x \text{ is an variable pattern}}{v \text{ matches } x \Rightarrow \{x \mapsto v\}}\ \textsc{varM} \qquad \frac{}{[] \text{ matches } \texttt{[]} \Rightarrow \varnothing}\ \textsc{nilM}$$

$$\frac{n \text{ matches } p_1 \Rightarrow \mathcal{E}_1 \qquad [n_1, \ldots, n_k] \text{ matches } p_2 \Rightarrow \mathcal{E}_2}{[n, n_1, \ldots, n_k] \text{ matches } p_1 \texttt{ :: } p_2 \Rightarrow \mathcal{E}_1 \cup \mathcal{E}_2}\ \textsc{consM}$$

$$\frac{\mathcal{E} \vdash v_1 \text{ matches } p_1 \Rightarrow \mathcal{E}_1 \qquad \ldots \qquad \mathcal{E} \vdash v_k \text{ matches } p_k \Rightarrow \mathcal{E}_k}{\mathcal{E} \vdash v_1 \texttt{ , } \ldots \texttt{ , } v_k \text{ matches } p_1 \texttt{ , } \ldots \texttt{ , } p_k \Rightarrow \mathcal{E}_1 \cup \cdots \cup \mathcal{E}_k}\ \textsc{tupleM}$$

$$\frac{\mathcal{E} \vdash e_0 \Downarrow v_0 \qquad \forall j < i (v_0 \text{ does not match } p_j) \qquad v_0 \text{ matches } p_i \Rightarrow \mathcal{E}' \qquad \mathcal{E}'' = \text{merge}(\mathcal{E}, \mathcal{E}') \qquad \mathcal{E}'' \vdash e_i \Downarrow v}{\mathcal{E} \vdash \texttt{match } e_0 \texttt{ with } \texttt{|}\ p_1 \texttt{ -> } e_1 \ldots \texttt{ |}\ p_k \texttt{ -> } e_k \Downarrow v}\ \textsc{matchE}$$

Summary

We can formally define the syntax, typing and semantics of **match expressions**

Deep pattern matching requires the introduction of **patterns**

Practice Problem

```
match 1 :: 2 :: [] with
| [] -> 0
| _ :: x :: [] -> x
```

Give typing and semantic derivations for the above expression (do you're best with space)

Exhaustiveness Checking

The Problem

```
match 1 :: 2 :: [] with
| [] -> 0
| _ :: x :: [] -> x
```

We'd like to know not just that our matches are well-typed, but also that we *cover all our bases*

Warnings for Pattern Matching

There are two kinds of warnings we can get:

» **Non-exhaustive.** There is a possible input that will not be matched

» **Useless.** There is a pattern that will never be used

We can determine both of these by a kind of *reachability analysis* which makes heavy use of *decision trees*

Useful Patterns

match e with

| p_1 → ...

⋮

| p_n →

$\vec{p}$

<u>Def</u>. A pattern q is <u>useful</u> w.r.t. $\vec{p}$ if there is a value v s.t.

1. v does not match $\vec{p}$] $\forall i.$ v does not match p_i
2. v does match q

<u>Notion:</u> $\mathcal{U}(\vec{p}, q) \equiv$ "q is useful w.r.t. $\vec{p}$"

(bool. func.)

Useful Patterns

$v = \text{Con}(1, \text{Nil})$

$$U([\text{Nil}], \text{Cons}(_, _)) = \text{true}$$

$v = \text{Cons}(1, \text{Nil})$

$$U\left(\left[\begin{array}{l}\text{Nil} \\ \text{Cons}(_, \text{Cons}(_, _))\end{array}\right], \text{Cons}(_, _)\right) = \text{true}$$

$$U\left(\left[\begin{array}{l}\text{Nil} \\ \text{Cons}(_, _)\end{array}\right], \text{Cons}(_, \text{Cons}(_, _))\right) = \text{false}$$

Exhaustiveness

<u>Def</u>. A pat. vec. $\vec{p}$ is exhaustive if

$$U(\vec{p}, _) = \text{false}$$

<u>ex</u>.

$$\begin{bmatrix} \text{Nil} \\ \text{Cons}(_, _) \end{bmatrix}$$ is exhaustive

$$\begin{bmatrix} \text{Nil} \\ \text{Cons}(_, \text{Nil}) \\ \text{Cons}(_, \text{Cons}(_, _)) \end{bmatrix}$$ is too.

$$\begin{bmatrix} \text{Nil} \\ \text{Cons}(_, \text{Cons}(_, _)) \end{bmatrix}$$ is <u>not</u>.

Uselessness

<u>Def</u>. Given pat. seq. $\vec{p}$, p_i is useless if

$$\mathcal{U}\left(\begin{bmatrix} p_1 \\ \vdots \\ p_{i-1} \end{bmatrix}, p_i\right) = \text{false}$$

<u>ex</u>.

$$\mathcal{U}\left(\begin{bmatrix} \text{Nil} \\ \text{Cons}(-,-) \end{bmatrix}, \text{Cons}(-, \text{Cons}(-,-))\right) = \text{false}$$

Restricted Setting

match ~~e~~ with

| p_1 → ~~e_1~~

⋮

| p_k → ~~e_k~~

$\vec{p}$ pattern vector

<pat> ::= _ | Nil | Con(<pat>, <pat>) | <pat>, ..., <pat>

wildcard

Useful Patterns (Recursive Definition)

assume well-typed

$U(\boxed{\vec{p}}, q)$

- p has wildcard → false
- no wilds ↓
- wild ↓
- tuple →
- constructors (Nil, Cons) ↓
  - Nil → $U(\vec{p}, \text{Nil})$
    $\vec{p}$ has no Nil
  - $\text{Cons}(q_1, q_2)$ → $U(\vec{p}, \text{Cons}(q_1, q_2))$
    $\parallel$
    $U(\vec{p}^{\,*}, (q_1, q_2))$

$$\begin{bmatrix} p_1 \\ \vdots \\ p_k \end{bmatrix} \begin{matrix} \to \text{Cons}(r_1, r_2) \\ \vdots \\ \to \text{Cons}(s_1, s_2) \\ \vdots \\ \to \text{Cons}(t_1, t_2) \end{matrix} \quad \vec{p}^{\,+} \begin{bmatrix} (r_1, r_2) \\ \vdots \\ (s_1, s_2) \\ \vdots \\ (t_1, t_2) \end{bmatrix}$$

Useful Patterns (Recursive Definition)

$U(\vec{p}, q)$

$\downarrow$ no wilds in p

$\vdots$

$\downarrow$ q is wild

$U(\vec{p}, _)$

list

$U(\vec{p}, \text{Nil})$

<u>or</u>

$U(\vec{p}, \text{Cons}(_, _))$

$$\vec{p} = \begin{bmatrix} \text{Nil} & \\ \text{Cons} & \cdots \\ \vdots & \\ \text{Cons} & \cdots \end{bmatrix}$$

triple

$$\begin{bmatrix} \underbrace{(_, \ldots, _)}_{\text{length } k} \\ \vdots \\ \underbrace{(_, \ldots, _)}_{\text{length } k} \end{bmatrix}$$

$U(\vec{p}, \underbrace{(_, \ldots, _)}_{\text{length } k})$

Useful Patterns (Recursive Definition)

$U(\vec{p}, q)$

$\downarrow$ no wilds in $\vec{p}$

q is $(q_1, q_2, \ldots, q_n)$

q_1 is :

const.

wild

tuple

$q_1 = (r_1, \ldots, r_k)$

$U(\vec{p}^*, r_1, \ldots, r_k, q_2, \ldots, q_n)$

$$\vec{p} = \begin{bmatrix} ((s_1, \ldots, s_k), \ldots) \\ ((t_1, \ldots, t_k), \ldots) \\ \vdots \\ ((l_1, \ldots, l_k), \ldots) \end{bmatrix} \Rightarrow \vec{p}^* = \begin{bmatrix} (s_1, \ldots, s_k, \ldots) \\ \vdots \\ (l_1, \ldots, l_k, \ldots) \end{bmatrix}$$

Useful Patterns (Recursive Definition)

$$U(\vec{p}, (\mathrm{Nil}, q_2, \ldots, q_k)) = U(\vec{p}^{*}, (q_2, \ldots, q_k))$$

$$\vec{p} = \begin{bmatrix} \mathrm{Nil}, & \vec{p}_1', \ldots \\ \vdots & \\ \mathrm{cons}(\ldots), & \vec{p}_2', \ldots \\ \vdots & \\ - & , \vec{p}_3', \ldots \\ & \vdots \end{bmatrix} \quad \begin{matrix} \vec{p}_1' \\ \\ \vec{p}_3' \end{matrix} \quad \begin{bmatrix} \vec{p}_1' \\ \vdots \\ \vec{p}_3' \end{bmatrix} = \vec{p}^{*}$$

$$U(\vec{p}, (\mathrm{Con}(r_1, r_2), q_2, \ldots, q_k)) = U(\vec{p}^{*}, r_1, r_2, q_2, \ldots, q_k)$$

$$\vec{p} \begin{bmatrix} -, & \vdots, & p_0' \\ \mathrm{Nil}, & \vec{p}_1' & \\ & \vdots & \\ \mathrm{Con}(l_1, l_2) & & p_2' \\ & \vdots & \end{bmatrix} \quad \begin{bmatrix} -, -, & \vec{p}_0' \\ & \vdots \\ l_1, l_2, & \vec{p}_2' \end{bmatrix} = \vec{p}^{*}$$

Examples

```
match l, r with
| Nil, _ → ...
| _, Nil → ...
```

$$\vec{p} = \begin{bmatrix} \text{Nil}, _ \\ _, \text{Nil} \end{bmatrix}$$

$$U(\vec{p}, \text{Cons}(_, _), \text{Cons}(_, _)) = \text{true}$$

Examples

$$u\left(\begin{bmatrix} Nil, & - \\ -, & Nil \end{bmatrix}, -\right) =$$

$$u\left(\begin{bmatrix} Nil, & - \\ -, & Nil \end{bmatrix}, (-, -)\right) =$$

$$u\left(\begin{bmatrix} Nil, & - \\ -, & Nil \end{bmatrix}, (Nil, -)\right)$$

$$\parallel$$

$$u\left(\begin{bmatrix} - \\ Nil \end{bmatrix}, -\right) = \text{false}$$

$Cons(-,-), Cons(-,-)$

$$u\left(\begin{bmatrix} \cancel{Nil}, & - \\ -, & Nil \end{bmatrix}, (Cons(-,-), -)\right)$$

$$\parallel$$

$$u\left((-, -, Nil), (- -, -)\right)$$

$$\vdots$$

$$u(Nil, -) = \text{true}$$

$Cons(-,-)$